

Exercise 3

Nashkerskyi Volodymyr 12407818

Avakian Valeriia 12432869

Dobrynina Ekaterina 12332276

VU Machine Learning

TU Wien, Austria



- Implementation of Simulated Annealing Algorithm for automated selection/configuration of machine learning algorithms
- Comparison of Simulated Annealing Algorithm execution with 4 classification datasets
- Implementation of two state of the art AutoML systems (auto-sklearn, TPOT)
- Comparison of metrics and execution time of Simulated Annealing Algorithm with existing AutoML tools



Logistic Regression

- Hyperparameters:
 - C: [0.0001, 0.001, 0.01, 0.1, 1.0, 10, 100]
 - solver: ["liblinear", "lbfgs"]

Random Forest

- Hyperparameters:
 - n_estimators: [5, 10, 20, 30, 50, 100, 200]
 - max_depth: [None, 5, 10, 15, 20, 25, 30]
 - criterion: ["gini", "entropy"]

Gradient Boosting

- Hyperparameters:
 - n_estimators: [5, 10, 20, 30, 50, 100, 200]
 - learning_rate: [0.00001, 0.0001, 0.001, 0.01, 0.1]
 - max_depth: [3, 5, 7, 9]

Support Vector Classifier (SVC)

- Hyperparameters:
 - C: [0.01, 0.1, 1.0, 10]
 - kernel: ["linear", "rbf"]
 - gamma: ["scale", "auto"]

K-Nearest Neighbors (KNN)

- Hyperparameters:
 - n_neighbors: [1, 3, 5, 7, 9]
 - weights: ["uniform", "distance"]
 - p: [1, 2] (Manhattan or Euclidean distance)



Generate a random solution from the defined search space.

```
def random_solution(search_space):  
    """  
    Generate a random solution from the defined search space.  
    A solution is a dict:  
    {  
        "algo_name": <string>,  
        "hyperparams": <dict with each param chosen randomly from the possible range>  
    }  
    """  
    algo_name = random.choice(list(search_space.keys()))  
    hyperparams_choices = search_space[algo_name]["hyperparams"]  
  
    chosen_hyperparams = {}  
    for param_name, possible_values in hyperparams_choices.items():  
        chosen_hyperparams[param_name] = random.choice(possible_values)  
  
    return {  
        "algo_name": algo_name,  
        "hyperparams": chosen_hyperparams  
    }
```



Simulated Annealing Algorithm: get neighbor

Produce a 'neighbor' solution by randomly change either the algorithm or one of the hyperparameters.

We define the 'neighbor' as:

- with 50% chance, switch the algorithm, otherwise, pick one hyperparameter and change it

```
def get_neighbor(solution, search_space):
```

```
    neighbor_sol = {
        "algo_name": solution["algo_name"],
        "hyperparams": solution["hyperparams"].copy()
    }
```

```
    # Decide whether we switch algorithm or tune a hyperparam
```

```
    if random.random() < 0.50:
```

```
        # Switch algorithm
```

```
        new_algo = random.choice(list(search_space.keys()))
```

```
        while new_algo == neighbor_sol["algo_name"]:
```

```
            new_algo = random.choice(list(search_space.keys()))
```

```
        # pick random hyperparams for the new algorithm
```

```
        hyperparams_choices = search_space[new_algo]["hyperparams"]
```

```
        chosen_hyperparams = {}
```

```
        for param_name, possible_values in hyperparams_choices.items():
```

```
            chosen_hyperparams[param_name] = random.choice(possible_values)
```

```
    neighbor_sol["algo_name"] = new_algo
```

```
    neighbor_sol["hyperparams"] = chosen_hyperparams
```

```
    else:
```

```
        # Switch one hyperparam from the same algorithm
```

```
        algo_name = neighbor_sol["algo_name"]
```

```
        hyperparams_choices = search_space[algo_name]["hyperparams"]
```

```
        param_to_change = random.choice(list(hyperparams_choices.keys()))
```

```
        current_value = neighbor_sol["hyperparams"][param_to_change]
```

```
        possible_values = hyperparams_choices[param_to_change]
```

```
        # pick a new value for that param (different from the current)
```

```
        new_value = random.choice(possible_values)
```

```
        while new_value == current_value and len(possible_values) > 1:
```

```
            new_value = random.choice(possible_values)
```

```
        neighbor_sol["hyperparams"][param_to_change] = new_value
```

```
    return neighbor_sol
```

Simulated Annealing Algorithm: building a model

Given a solution (algorithm + hyperparams), build and return the corresponding sklearn model

```
def build_model(solution, search_space):  
    algo_name = solution["algo_name"]  
    hp = solution["hyperparams"]  
    return search_space[algo_name]["build_fn"](hp)
```



Evaluate a solution by training via cross-validation

The evaluation metric is accuracy

```
def evaluate_solution(solution, X, y, cv_folds=5):  
  
    model = build_model(solution, search_space)  
    cv = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=42)  
    scores = cross_val_score(model, X, y, cv=cv, scoring='accuracy', n_jobs=-1)  
    mean_score = np.mean(scores)  
  
    # We transform the objective so that "lower is better" --> negative accuracy  
    # Alternatively, we can keep it as 1 - accuracy, or any cost function you prefer.  
    cost = -mean_score  
    return cost
```



Simulated Annealing Algorithm

Perform Simulated Annealing to find best (algorithm, hyperparameters) that maximizes accuracy (or equivalently minimizes negative accuracy)

```
def simulated_annealing(  
    X, y,  
    search_space,  
    max_iterations=50,  
    initial_temperature=1.0,  
    min_temperature=0.001,  
    alpha=0.85,  
    inner_loop=10,  
    time_limit_minutes=70,  
    random_seed=42  
):
```

- X, y: features, labels
- search_space: dict describing algorithms and hyperparams
- max_iterations: how many outer iterations (cooling steps)
- initial_temperature: starting "temperature"
- min_temperature: minimal temperature to stop
- alpha: cooling ratio $T \leftarrow \alpha * T$
- inner_loop: how many neighbor solutions to try at each temperature
- random_seed: for reproducibility




```
def simulated_annealing(
    ...
):
    current_solution = random_solution(search_space)
    current_cost = evaluate_solution(current_solution, X, y)
    best_solution = current_solution
    best_cost = current_cost

    T = initial_temperature
    iteration = 0
    start_time = time.time()
    time_limit_seconds = time_limit_minutes * 60
    # Start SA loop
    while T > min_temperature and iteration < max_iterations:
        elapsed_time = time.time() - start_time
        if elapsed_time > time_limit_seconds:
            print(f"Time limit exceeded. Stopping optimization. Best Cost: {best_cost:.4f}")
            break
        print(f"Iteration {iteration + 1}/{max_iterations} - Temperature: {T:.4f} - Current Cost: {current_cost:.4f} - Best Cost: {best_cost:.4f}")

        < inner loop >

        # cool down
        T = alpha * T
        iteration += 1

    return best_solution, best_cost
```



Inner loop, start tracking time

```
for inner_iter in range(inner_loop):
    # get neighbor
    neighbor = get_neighbor(current_solution, search_space)
    neighbor_cost = evaluate_solution(neighbor, X, y)

    # if neighbor is better, accept it
    if neighbor_cost < current_cost:
        current_solution = neighbor
        current_cost = neighbor_cost
        print(f" Inner {inner_iter + 1}/{inner_loop}: Accepted better solution with cost: {current_cost:.4f}")
    else:
        # accept with probability  $e^{-(neighbor\_cost - current\_cost)/T}$ 
        cost_diff = neighbor_cost - current_cost
        acceptance_prob = np.exp(-cost_diff / T)
        if random.random() < acceptance_prob:
            current_solution = neighbor
            current_cost = neighbor_cost
            print(f" Inner {inner_iter + 1}/{inner_loop}: Accepted worse solution with cost: {current_cost:.4f} (prob: {acceptance_prob:.4f})")

    # update global best if needed
    if current_cost < best_cost:
        best_solution = current_solution
        best_cost = current_cost
        print(f" New Best Found: Cost: {best_cost:.4f}, Algorithm: {best_solution['algo_name']}")
```



The datasets differ in terms of dimensions, classes (binary vs. multiclass)

Datasets overview:

- Loan dataset shape: (7000, 180)
- Census Income dataset shape: (24542, 7)
- Bank Marketing dataset shape: (31647, 16)
- Student Dropout dataset shape: (3096, 94)



Results: Simulated Annealing

	Simulated Annealing			
	Loan shape (7000, 180)	Census Income shape (24542, 7)	Bank Marketing shape (31647, 16)	Student Dropout shape (3096, 94)
N of iterations	13	16	16	66
Algorithm	gradient boosting	gradient boosting	gradient boosting	gradient boosting
Hyperparameters	n_estimators: 200, learning_rate: 0.1, max_depth: 5	n_estimators: 200, learning_rate: 0.1, max_depth: 5	n_estimators: 200, learning_rate: 0.1, max_depth: 3	n_estimators: 100, learning_rate: 0.1, max_depth: 3
Accuracy	0,989	0,989	0,905	0,778
Total runtime (sec)	4671	4274	4324	4217



Results: AutoML

	AutoSklearn			
	Loan shape (7000, 180)	Census Income shape (24542, 7)	Bank Marketing shape (31647, 16)	Student Dropout shape (3096, 94)
N of iterations				
Algorithm	liblinear_svc	gradient_boosting	random_forest	liblinear_svc
Hyperparameters				
Accuracy	0,996	0,892	0,953	0,902

	TPOT			
	Loan shape (7000, 180)	Census Income shape (24542, 7)	Bank Marketing shape (31647, 16)	Student Dropout shape (3096, 94)
N of iterations	15	10	10	20
Algorithm	random forest	gradient boosting	random forest	random forest
Hyperparameters	criterion: 'entropy', max_features = 0.2, min_samples_leaf = 2, min_samples_split = 4	learning_rate = 0.5, max_depth = 8, max_features = 0.7, min_samples_leaf = 6	max_features = 0.45, min_samples_leaf = 8, min_samples_split = 4	bootstrap: False, max_features = 0.2, min_samples_leaf = 3, min_samples_split = 10
Accuracy	0,973	0,908	0,939	0,951

Total runtime (sec)

9921

13507

11464

5970



Comparison vs AutoML

AutoSklearn (0.996) performed best, but Simulated Annealing (0.989) and TPOT (0.973) also showed high accuracy

Loan
shape (7000, 180)

	Simulated Annealing	AutoSklearn	TPOT
N of iterations	13		15
Algorithm	gradient boosting	liblinear_svc	random forest
Hyperparameters	n_estimators: 200, learning_rate: 0.1, max_depth: 5		criterion: 'entropy', max_features = 0.2, min_samples_leaf = 2, min_samples_split = 4
Accuracy	0,989	0,996	0,973
Total runtime (sec)	4671		9921



Comparison vs AutoML

TPOT (0.908) slightly outperforms Simulated Annealing (0.905), while AutoSklearn shows (0.892)

Census Income
shape (24542, 7)

	Simulated Annealing	AutoSklearn	TPOT
N of iterations	16		10
Algorithm	gradient boosting	gradient_boosting	gradient boosting
Hyperparameters	n_estimators: 200, learning_rate: 0.1, max_depth: 5		learning_rate = 0.5, max_depth = 8, max_features = 0.7, min_samples_leaf = 6
Accuracy	0,989	0,892	0,908
Total runtime (sec)	4274		13507



Comparison vs AutoML

AutoSklearn (0.953) outperforms TPOT (0.939) and Simulated Annealing (0.905)

Bank Marketing shape (31647, 16)			
	Simulated Annealing	AutoSklearn	TPOT
N of iterations	16		10
Algorithm	gradient boosting	random_forest	random forest
Hyperparameters	n_estimators: 200, learning_rate: 0.1, max_depth: 3		max_features = 0.45, min_samples_leaf = 8, min_samples_split = 4
Accuracy	0,905	0,953	0,939
Total runtime (sec)	4324		11464



Comparison vs AutoML

TPOT (0.951) has the best accuracy, then AutoSklern (0.902) and Simulated Annealing (0.778)

Student Dropout shape (3096, 94)			
	Simulated Annealing	AutoSklern	TPOT
N of iterations	66		20
Algorithm	gradient boosting	liblinear_svc	random forest
Hyperparameters	n_estimators: 100, learning_rate: 0.1, max_depth: 3		bootstrap: False, max_features = 0.2, min_samples_leaf = 3, min_samples_split = 10
Accuracy	0,778	0,902	0,951
Total runtime (sec)	4217		5970

Simulated Annealing does not cope with this dataset in terms of accuracy



- Despite the limited search space and time (1 hour 10 minutes), SA demonstrates **competitive results**
- It's a coincidence, but SA chose Gradient Boosting with minimal changes in hyperparameters for all 4 datasets
- With Loan Dataset, SA shows almost the **same accuracy** (0.989) as AutoSklearn (0.996) and TPOT (0.973), but **runs 2 times faster**
- Even **AutoML systems can make mistakes** in choosing a model and hyperparameters - on Census Income, TPOT and SA choose Gradient Boosting, showing similar results (0.905 and 0.908), while AutoSklearn uses Gradient Boosting with a different hyperparameter setting and shows (0.892).
- TPOT is effective for obtaining high-accuracy solutions, but takes significantly longer than AutoSklearn
- TPOT showed a variety of models and hyperparameters on 4 datasets, but it increases the execution time significantly - TPOT is beneficial for problems where model versatility is important



- Mismatch of library versions
- Lack of resources for large datasets: a dataset with 25,000 rows and using Random Forest with `n_estimators=200` requires a significant amount of RAM and time. On MacOS, especially on laptops, this leads to system overload or long calculations
- Multiple warnings make the analysis process less convenient
- Using Docker improves stability but increases the complexity of setting up the environment



Simulated Annealing:

- Requires manual parameter tuning (alpha, max_iterations, inner_loop, etc.)
- More complex to implement, but allows full control over the optimization process
- Requires significant time and resources to execute
- The results show that SA can find sufficiently good hyperparameters for models
- More transparent and controllable process (it is possible to explicitly set the rules and the space)
- Possibility to study the influence of specific hyperparameters on the result

AutoML:

- Easier to use
- Shorter execution time, but may be limited to simpler models
- Often shows higher accuracy, as they use complex approaches to find a globally optimal solution
- Has an automated process for building pipelines

Conclusion: Simulated Annealing is suitable for users who are ready to fine-tune the algorithm, while Auto-Sklearn is convenient for quick and automatic application



During the current task, we

- learned how to apply Simulated Annealing to ML
- studied how SA can be adapted for algorithm selection and hyperparameter optimization
- understood SA limitations (runtime, dependence on the initial solution) and advantages (flexibility and ease of implementation)
- became familiar with state of the art AutoML tools: auto-sklearn, TPOT
- convinced that AutoML is better suited for complex problems that require a fast and high-quality solution

